Chess Playing Model using Soft Computing

SCI Research Papers, Architecture & Training Data

Subject: Soft Computing | Techniques: Genetic Algorithm + Fuzzy Logic

Part 1 — Five SCI Research Papers

1. Genetic Algorithms for Evolving Computer Chess Programs

David, van den Herik, Koppel & Netanyahu — *IEEE Transactions on Evolutionary Computation*, 2014 (SCIE)

Uses a genetic algorithm to automatically learn the weights of a chess evaluation function, instead of a human setting them by hand. The evolved program reached grandmaster strength. This is the main paper our project is based on.

Journal page: https://ieeexplore.ieee.org/document/6626616

FREE FULL PDF: https://arxiv.org/pdf/1711.08337

2. An Evolutionary Algorithm with a History Mechanism for Tuning a Chess Evaluation Function

Vázquez-Fernández, Coello Coello & Sagols Troncoso — *Applied Soft Computing*, 2013 (SCIE)

Tunes the evaluation function by checking how many grandmaster moves the program can correctly guess. This gives us a fast and simple way to score each candidate during training.

Journal page: https://www.sciencedirect.com/science/article/abs/pii/S1568494613000690

PAID — use college library login, or request the PDF free from the authors on ResearchGate.

3. History Mechanism Supported Differential Evolution for Chess Evaluation Function Tuning

Bošković, Brest, Zamuda, Greiner & Žumer — *Soft Computing* (Springer), 2011 (SCIE)

Uses another soft-computing method (differential evolution) to tune the chess evaluation weights. Tuning improved the engine's strength by about 155 rating points, showing how much this approach helps.

Journal page: https://link.springer.com/article/10.1007/s00500-010-0593-z

PAID — use college library login, or request the PDF free from the authors on ResearchGate.

4. A Self-Learning Evolutionary Chess Program

Fogel, Hays, Hahn & Quon — *Proceedings of the IEEE*, 2004 (SCIE)

A chess program ("Blondie25") that learns to play on its own by combining an evolutionary algorithm with small neural networks, using only the basic rules. Its strength improved by nearly 400 rating points.

Journal page: https://doi.org/10.1109/JPROC.2004.837633

PAID — use college library login, or request the PDF free from the authors on ResearchGate.

5. A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play

Silver et al. (AlphaZero) — *Science*, 2018 (SCI)

The famous AlphaZero paper. It learns chess from scratch by playing against itself, using a neural network and tree search, and beat the strongest engine of its time. Included as the state-of-the-art reference for learning-based chess.

Journal page: https://www.science.org/doi/10.1126/science.aar6404

FREE FULL PDF (author preprint version): https://arxiv.org/pdf/1712.01815

Note on getting the paid papers. Papers 2, 3 and 4 are behind publisher paywalls, so they cannot be downloaded freely. Three legal ways to get them: (1) log in through your college library, which usually has IEEE, Springer and Elsevier subscriptions; (2) open the paper on ResearchGate and click "Request full-text" — authors almost always send it; (3) email the first author directly. For the report itself, the abstract plus the summary above is normally enough to cite the paper.

Part 2 — Proposed Architecture

2.1 Overview

Our project is a chess program that a human can play against. It has six parts. The board is read in, all legal moves are found, the program looks a few moves ahead, and an **evaluation function** gives each position a score. The two soft-computing techniques sit around this evaluation function:

- **Genetic Algorithm** `GA` — automatically learns the best weights for the evaluation function, instead of a human guessing them.
- **Fuzzy Logic** `FUZZY` — decides how much the game is in the opening, middlegame or endgame, and smoothly adjusts those weights to suit the current phase.

2.2 Architecture Diagram



2.3 Explanation of Each Module

Module 1 — Board Input

This part reads the current chess position. From the position it pulls out a set of *features* — simple numbers that describe how good or bad the position is. Examples: how many pieces each side has (material), how many squares the pieces can move to (mobility), how safe each king is, and whether the pawns are well arranged. These features are the `f1, f2, f3 ...` values used later in the evaluation function.

Module 2 — Move Generator

Finds every legal move available in the current position, and also detects check, checkmate, stalemate and draws. We use the ready-made *python-chess* library for this, because chess rules (castling, en passant, promotion) are complicated and writing them ourselves would waste time without adding anything to the project.

Module 3 — Search Module

This part looks ahead. It uses **minimax**: our program assumes it will play the best move, and the opponent will reply with their best move, and so on for a few turns. **Alpha-beta pruning** is added to skip branches that cannot possibly be better, which makes the search much faster. We search about 4 to 6 moves deep, which runs comfortably on a normal laptop. At the end of each line, the position is sent to the evaluation function to be scored.

Module 4 — Evaluation Function

This is the heart of the program. It converts a chess position into a single number that says who is better and by how much. It works as a weighted sum:

> **score = w1×f1 + w2×f2 + w3×f3 + … + wn×fn**
> where *f* are the position features from Module 1, and *w* are the weights that decide how important each feature is.

A positive score means White is better, negative means Black is better. The whole difficulty is choosing good weights — and that is exactly what our two soft-computing modules do.

Module 5 — Genetic Algorithm SOFT COMPUTING

Normally a programmer guesses the weights by hand. Instead, we let a genetic algorithm learn them. It works like natural selection:

1. **Chromosome:** one complete set of weights `[w1, w2, ... wn]` is treated as one "individual".
2. **Population:** we start with 30–50 random individuals.
3. **Fitness:** each individual is scored on how well it plays — either by playing games, or by checking how many grandmaster moves it guesses correctly (explained in Part 3).
4. **Selection:** the better-scoring individuals are chosen to become parents.
5. **Crossover:** two parents are combined to make a new set of weights.
6. **Mutation:** small random changes are added so new ideas can appear.
7. **Repeat** for 30–100 generations. The best weight set at the end, `w*`, is what our final engine uses.

Module 6 — Fuzzy Logic Module SOFT COMPUTING

A single fixed set of weights is not ideal, because good chess changes as the game goes on. In the middlegame the king must stay safe behind its pawns; in the endgame the same king should walk into the centre and become active. So the weights should change with the phase of the game.

A simple hard rule like "*if fewer than 10 pieces then it is the endgame*" is crude — the game does not suddenly change at one exact piece. Fuzzy logic solves this by letting a position be, for example, 30% middlegame and 70% endgame at the same time, and blending the weights smoothly.

Stage	What happens
Inputs	Two values: total material left on the board, and the current move number.
Fuzzification	Each input is converted into fuzzy terms. Material becomes LOW / MEDIUM / HIGH. Move number becomes EARLY / MIDDLE / LATE.
Fuzzy rules	IF material is HIGH AND moves are EARLY THEN phase is OPENING. IF material is MEDIUM AND moves are MIDDLE THEN phase is MIDDLEGAME. IF material is LOW THEN phase is ENDGAME.
Output	A degree of membership for each phase, for example: opening 0.0, middlegame 0.3, endgame 0.7.
Effect on weights	These degrees blend between phase-specific weight sets. Example: king-safety weight is high in the middlegame set and low in the endgame set, so at 0.3 / 0.7 the final king-safety weight is mostly the endgame value — a smooth transition, not a sudden jump.

> **How the two techniques work together:** the Genetic Algorithm decides *what the weights should be*, and the Fuzzy Logic module decides *how those weights should shift as the game progresses*. The evaluation function then uses the final, phase-adjusted weights to score every position.

2.4 Data Flow (step by step)

1. The human plays a move on the Pygame board.
2. Module 1 reads the new position and extracts its features.
3. Module 2 generates all legal replies.
4. Module 3 searches a few moves ahead through those replies.
5. For every position reached, Module 6 (fuzzy) works out the game phase and adjusts the weights, and Module 4 uses those weights to calculate a score.
6. Module 3 picks the move leading to the best score and plays it.
7. Module 5 (the genetic algorithm) is run *before* the game, during a training phase, to produce the weights that Modules 4 and 6 use.

2.5 Tools and Prerequisites

Purpose	Tool
Programming language	Python 3
Chess rules and legal moves	python-chess
Mathematical operations	NumPy
Genetic algorithm	DEAP
Fuzzy logic system	scikit-fuzzy
Board interface to play on	Pygame
Testing engine strength	Stockfish (free)

Installation: `pip install python-chess numpy deap scikit-fuzzy pygame`

Part 3 — Training Data

3.1 Do we need data at all?

It depends on how we score each individual in the genetic algorithm. There are two options:

Method	How fitness is measured	Data needed?
Option 1 — Self-play	Individuals play games against each other. The ones that win more games get a higher fitness.	**No external data.** Games are generated by the program itself. But it is slow — thousands of games per generation.
Option 2 — Grandmaster move matching RECOMMENDED	Show the program a position from a real game and ask what move it would play. If it matches the move the grandmaster actually played, that is one point. Fitness = total points.	**Yes — a set of real game positions.** Much faster than self-play, and this is the method used in Paper 2.

We recommend **Option 2**, because it runs fast enough to finish on a laptop and it gives us a published paper to cite for the method.

3.2 Where to get the data (all free)

Source	Link	What it gives
Lichess Open Database (recommended)	https://database.lichess.org/	Over 8 billion rated games in PGN format, released as one file per month, completely free and open. Also has a separate puzzle database in CSV format.
Lichess Puzzle Database	https://database.lichess.org/#puzzles	A CSV file where every row already contains a position (FEN) and the correct move. Easiest starting point — no PGN parsing needed.
KingBase	Search "KingBase chess database"	Millions of games played by strong titled players, in PGN.
CCRL	https://computerchess.org.uk/ccrl/	Games played between chess engines, useful for testing.

Important practical warning. The monthly Lichess game files are very large — roughly 20 GB compressed, and about 7 times bigger once extracted. Do not download a whole month. Two easy ways around this: (1) the compressed files can be partly downloaded and then cancelled, and the partial file still works; or (2) just use the **puzzle CSV**, which is far smaller and already in position + correct move form. For a college project, **10,000 to 50,000 positions is more than enough**.

3.3 What format is the data in?

The data goes through three formats before it reaches our model.

Format 1 — PGN (the raw download)

PGN means Portable Game Notation. It is an ordinary **text file** that lists whole games move by move, which you can open in Notepad. It looks like this:

```
[White "Carlsen, Magnus"]
[Black "Nakamura, Hikaru"]
[Result "1-0"]

1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. O-O Be7 ...
```

Format 2 — FEN (one single position)

A PGN game is split into individual positions. Each position is written as a FEN string — again just **plain text**, one line that fully describes the board:

```
rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
```

Format 3 — Feature vector (what actually enters the model)

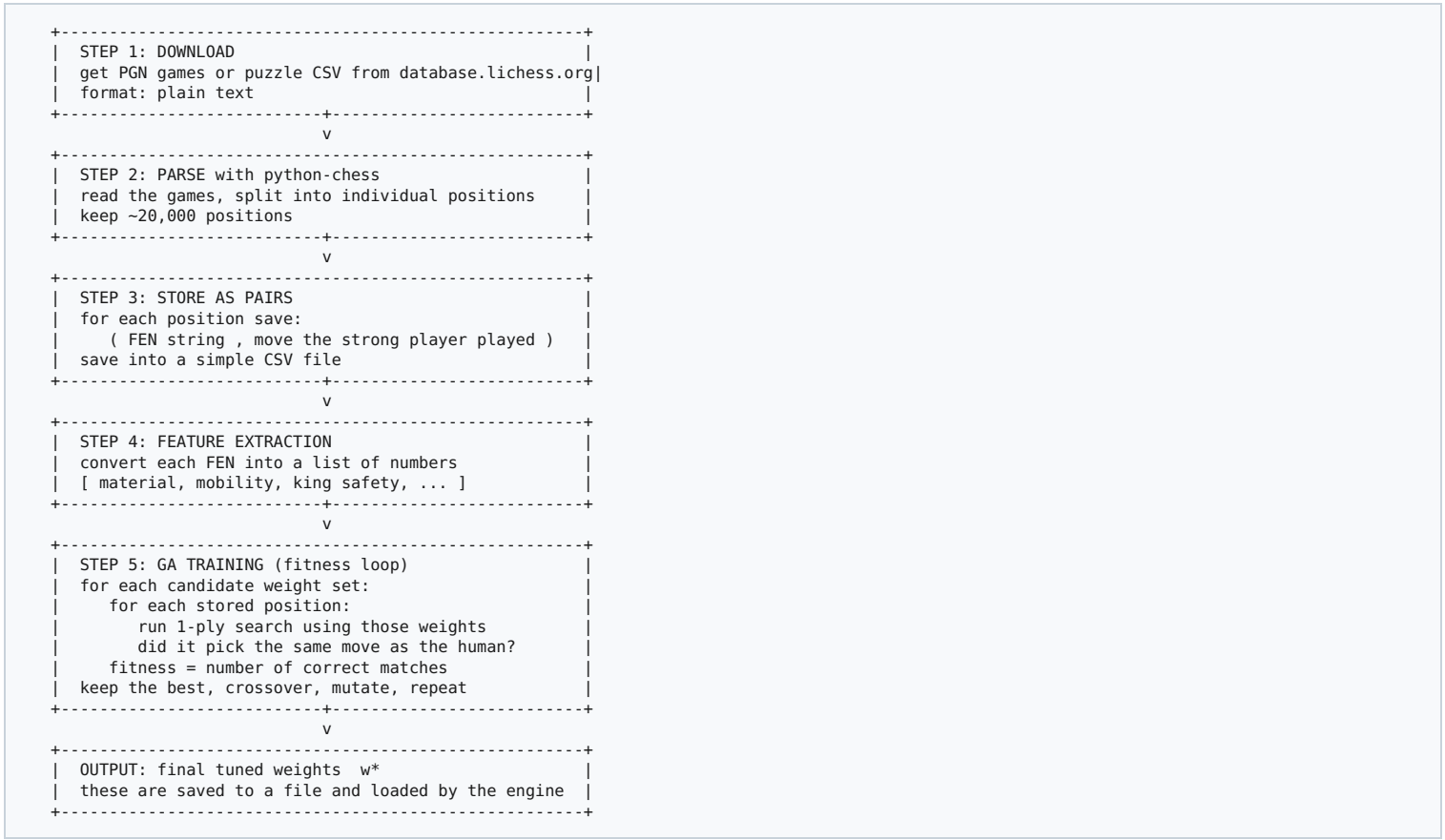
The FEN text is converted into a **list of plain numbers**. This numeric list is what the evaluation function multiplies by the weights. For example:

```
[ 3, 12, -1, 0, 5, 2, ... ]
   |   |   |   |   |   |
   |   |   |   |   |   +-- centre control
   |   |   |   |   +------ passed pawns
   |   |   |   +---------- doubled pawns
   |   |   +-------------- king safety
   |   +------------------ mobility (number of legal moves)
   +---------------------- material difference
```

Answer to "binary or other format?" For our project the data is fed in as **ordinary numbers, not binary**. The pipeline is: text (PGN) → text (FEN) → a list of numbers (feature vector) → evaluation function.

Binary encoding is only needed for the *neural network* approach (like AlphaZero in Paper 5), where the board is turned into a grid of 0s and 1s. Since we are using a genetic algorithm and fuzzy logic instead of a neural network, we do not need that. (Note: *bitboards* are binary and are used inside python-chess to make move generation fast, but that is internal to the library — it is not data we prepare or feed in.)

3.4 Complete data pipeline



3.5 Summary

Question	Answer
What data do we train on?	Real chess games played by strong players — specifically, positions taken from those games together with the move that was actually played.
Where do we get it?	Lichess Open Database (database.lichess.org) — free and open. The puzzle CSV is the easiest option.
How much do we need?	About 10,000–50,000 positions. Do not download the full 20 GB monthly file.
What format is it fed in?	As a list of ordinary numbers (a feature vector), not binary. Text PGN → text FEN → numeric features.
What is the model learning?	Not the moves themselves — it is learning the *weights* of the evaluation function, so that its own choices come to match those of strong players.

3.6 Expected Output and Testing

The final deliverable is a working chess program with a Pygame board that a human can play against. To show the soft-computing part actually worked, we compare two versions of the same engine: an **untuned version** with weights set by hand, and a **tuned version** with weights learned by the genetic algorithm and adjusted by fuzzy logic. Both play a fixed number of games against each other and against Stockfish at a limited level, and we record wins, draws, losses and the rating improvement. Paper 3 reports a gain of roughly 155 rating points from this kind of tuning, which is our reference target.